

Cahier des charges — Application Flutter 100% hors-ligne (Téléphones + Routeur)

Titre : Système de collecte et centralisation de commandes sur réseau intranet local (super-admin mobile)

Version : 1.0

Auteur : Prestige ZONDOGA / Conception technique

Date : 9 décembre 2025

1. Résumé exécutif

Ce document décrit, pas à pas, la conception, l'installation, le déploiement, les tests et la maintenance d'une application mobile Flutter fonctionnant entièrement hors-ligne sur un réseau local (intranet) composé uniquement de téléphones mobiles connectés à un routeur. Un appareil jouera le rôle de **Super-Admin** (nœud central) avec une IP statique obtenue via réservation DHCP ou configuration manuelle. Les autres appareils sont des **clients** qui enregistrent des commandes localement (SQLite) puis les transmettent vers le Super-Admin via HTTP local ou WebSocket.

Objectifs principaux : - Permettre la saisie et la collecte de commandes sur plusieurs téléphones. - Centraliser les commandes sur le Super-Admin sans Internet. - Assurer résilience, idempotence, et sécurité minimale adaptée à un réseau fermé.

2. Périmètre

Inclus : - App Flutter installée sur chaque téléphone (client) avec base SQLite. - App Flutter ou serveur léger sur le Super-Admin pour réception et stockage central (SQLite). - Routeur Wi-Fi local (sans accès Internet requis). - Communication via HTTP local et/ou WebSocket.

Exclus : - Services cloud, serveurs Internet, notifications push externes, intégration avec systèmes tiers.

3. Matériel & logiciels requis

Matériel

- N téléphones Android (iOS possible selon restrictions) avec Wi-Fi. Recommandé : Android 10+.
- 1 téléphone (ou Raspberry Pi / mini-PC) pour Super-Admin — **recommandation forte : Raspberry Pi** pour stabilité.
- 1 routeur Wi-Fi local (capable de DHCP & réservation d'IP). Option : hotspot dédié si pas de routeur.
- Câbles & chargeurs.

Logiciels

- Flutter SDK (pour développement)
 - `sqflite` / `drift` dans l'app Flutter
 - `http`, `web_socket_channel`, `uuid`, `connectivity_plus`
 - (optionnel) Node.js / Python si serveur central non Flutter
-

4. Rôles & responsabilités

- **Administrateur système** : configure le routeur, réserve IP, installe le Super-Admin.
 - **Développeur** : implémente l'app Flutter client et le serveur Super-Admin, tests.
 - **Utilisateur final** : saisit commandes dans l'app cliente.
-

5. Architecture technique (version rapide)

- Topologie : Étoile. Routeur au centre. Super-Admin (IP fixe) connecté au routeur. Clients connectés au même réseau.
 - Transport : HTTP POST JSON pour ingestion ; WebSocket pour push (optionnel).
 - Stockage : SQLite local sur chaque appareil ; DB centrale (SQLite/ PostgreSQL) sur Super-Admin.
-

6. Définition des API locales et schémas (simplifié)

Endpoint(s) (exemples) : - `POST http://<IP_SUPER_ADMIN>:8080/ingest` — recevoir un message JSON (order/event) - `GET http://<IP_SUPER_ADMIN>:8080/changes?since=<ts>` — récupérer changements depuis `ts` - `GET http://<IP_SUPER_ADMIN>:8080/status` — état du serveur - WS `ws://<IP_SUPER_ADMIN>:8080/ws` — WebSocket pour push en temps réel

Message type (JSON) :

```
{
  "message_id": "uuid-v4",
  "type": "order",
  "device_id": "uuid-device",
  "timestamp": "2025-12-09T12:00:00Z",
  "payload": { ... }
}
```

7. Plan détaillé étape par étape

Phase 0 — Préparation

1. **Choisir le Super-Admin** : définir quel téléphone sera Super-Admin ou préférer un Raspberry Pi.

2. **Préparer le routeur** : activer Wi-Fi, désactiver l'accès Internet si besoin, noter le sous-réseau (ex : `192.168.1.0/24`).
 3. **Réservation IP** : soit configurer IP statique sur l'appareil Super-Admin (par ex. `192.168.1.10`), soit faire une réservation DHCP dans l'interface du routeur.
 4. **Lister les appareils** : prévoir pour chaque téléphone un identifiant unique (UUID) et un nom.
-

Phase 1 — Conception & spécifications

1. **Modèle de données** : créer script SQL pour `devices`, `orders`, `sync_queue`, `logs`.
 2. **Définir scénario d'usage** : création commande, modification (si autorisée), suppression (si autorisée), sync.
 3. **Définir règles de gestion** : idempotence (`message_id` unique), horodatage UTC, taille max des messages.
 4. **Plan de résilience** : file d'attente persistante, retries, backoff.
-

Phase 2 — Développement (prototype)

1. **Créer projet Flutter** : structure multi-modules si besoin (client & server code partageable).
 2. **Implémenter modèle SQLite** : tables et DAO (utiliser `drift` pour productivité).
 3. **Implémenter queue locale** : table `sync_queue` + service `SyncService`.
 4. **Implémenter client HTTP** : méthode `sendToServer(payload)` avec gestion d'erreurs.
 5. **Implémenter serveur minimal sur Super-Admin** : route `/ingest` qui valide, stocke, renvoie ack.
 6. **Tester localement** : lancer serveur Super-Admin sur port 8080, envoyer requêtes POST depuis client.
-

Phase 3 — Tests unitaires & d'intégration

1. **Tests unitaires** sur DAO et logique de queue.
 2. **Tests d'intégration** : plusieurs émulateurs sur PC ou téléphones réels sur même réseau pour simuler charges.
 3. **Tests de robustesse** : couper réseau, redémarrer Super-Admin, vérifier retries et absence de doublons.
 4. **Tests de performance** : 50-100 messages/minute (ou selon besoin) pour vérifier comportement.
-

Phase 4 — Déploiement sur site (procédure pas-à-pas)

1. **Configurer le routeur** : définir SSID, mot de passe, désactiver accès Internet (optionnel), réserver IP Super-Admin.
2. **Configurer Super-Admin (téléphone)** : définir IP statique dans paramètres Wi-Fi si possible, ou vérifier réservation DHCP.
3. **Installer l'app sur Super-Admin** : installer build Flutter (APK) en mode production.
4. **Lancer le serveur sur Super-Admin** : démarrer l'app en mode receiver; vérifier `GET /status` répond.
5. **Installer l'app sur chaque client** : installer APK sur chaque téléphone.

6. **Configuration client** : entrer manuellement l'IP du Super-Admin (ou découvrir via mDNS si implémenté).
 7. **Vérifier connectivité** : `ping` Super-Admin depuis client (ou simple requête `GET /status`).
 8. **Faire test de bout en bout** : créer commande sur client, vérifier arrivée sur Super-Admin et ack au client.
-

Phase 5 — Validation & recette

1. **Procédure de recette** avec checklist : création, duplication, conflit, offline → online, restauration.
 2. **Validation utilisateur** : sessions de tests avec utilisateurs finaux.
 3. **Corrections** : corriger bugs remontés pendant recette.
-

Phase 6 — Mise en production & maintenance

1. **Livraison** : déployer version stable sur tous les téléphones.
 2. **Mode opératoire** : documenter comment redémarrer Super-Admin, récupérer logs, restaurer DB.
 3. **Backup** : prévoir export régulier de la DB centrale (USB, export CSV via UI).
 4. **Support** : définir contact et procédure pour incidents.
-

8. Scripts et snippets utiles (annexes)

8.1 SQL (création de tables)

```
CREATE TABLE devices (  
  device_id TEXT PRIMARY KEY,  
  name TEXT,  
  last_seen DATETIME  
);  
  
CREATE TABLE orders (  
  id TEXT PRIMARY KEY,  
  device_id TEXT,  
  created_at DATETIME,  
  status TEXT,  
  payload TEXT,  
  synced INTEGER DEFAULT 0  
);  
  
CREATE TABLE sync_queue (  
  queue_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  message_id TEXT,  
  payload TEXT,  
  attempts INTEGER DEFAULT 0,
```

```
    last_attempt DATETIME
);
```

8.2 Exemple serveur minimal Dart (shelf)

```
import 'dart:convert';
import 'package:shelf/shelf.dart';
import 'package:shelf/shelf_io.dart' as io;

void main() async {
  final handler = (Request req) async {
    if (req.method == 'POST' && req.requestedUri.path == '/ingest') {
      final body = await req.readAsString();
      final jsonBody = jsonDecode(body);
      print('Reçu: $jsonBody');
      // TODO : valider et stocker dans SQLite
      return Response.ok(jsonEncode({'status': 'ok'}), headers: {'content-type': 'application/json'});
    }
    return Response.notFound('Not Found');
  };

  await io.serve(handler, '0.0.0.0', 8080);
  print('Server running on port 8080');
}
```

9. Sécurité et bonnes pratiques

1. **Authentification simple** : token static (API_KEY) dans header pour autoriser clients.
2. **Validation d'entrée** : vérifier schéma JSON et taille.
3. **Limiter accès réseau** : réseau Wi-Fi fermé sans Internet et mot de passe fort.
4. **Journalisation** : conserver logs d'ingest et d'erreurs.

10. Planning indicatif (sprint)

- Jour 1-3 : Conception & modélisation DB.
 - Jour 4-10 : Développement prototype client + serveur.
 - Jour 11-14 : Tests unitaires & intégration.
 - Jour 15-17 : Déploiement pilote sur 3-5 téléphones.
 - Jour 18-21 : Corrections & recette utilisateur.
 - Jour 22 : Mise en production locale.
-

11. Livrables

- APK(s) Flutter : client et Super-Admin
 - Script SQL `create_tables.sql`
 - Guide d'installation pas-à-pas (routeur + appareils)
 - Rapport de tests et checklist de recette
-

12. Questions ouvertes / décisions à prendre

1. Super-Admin : téléphone ou Raspberry Pi ? (Recommandé : Raspberry Pi)
 2. Méthode de découverte d'IP : statique ou mDNS ?
 3. Nécessité d'un chiffrement TLS local ?
 4. Volume estimé de messages / fréquence pour dimensionner tests.
-

Si tu veux, j'ajoute maintenant : - le **script SQL complet** prêt à exécuter, - un **APK Flutter minimal** (code complet) avec client + serveur, - ou le **guide d'installation pour Raspberry Pi** (Node.js + sqlite).

Dis-moi lequel tu veux en premier.